PROGRAMMING IN JAVA ASSIGNMENT 3 – CO3

CO3 Coverage: Exception handling – exception hierarchy, checked and unchecked exceptions, try, catch and finally, throw and throws, user-defined exceptions, exception chaining and propagation, try-with-resources and debugging; Multithreading – thread life cycle, Thread and Runnable, thread methods, synchronization, deadlock, inter-thread communication, the Executor framework, Callable and Future, and concurrent collections (Sessions 27–38).

Mode: Pair Work – 2 Students per Group

Duration: 1 Week

Total Marks: 50

Submission Date: _____

Bloom's Levels Assessed: K4 – Analyse and K5 – Evaluate (the level of each task is stated under the task heading)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs. In this pair-work assignment, **both students must execute every question in their own repository**, not only the part for which they are primarily responsible.
- Create a GitHub repository named **CSA09-<RegisterNumber>-Assignment3** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the Java source code, input files and output screenshots.
- Include a README.md with your name, register number, Java version, the steps to compile and run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.

- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link : https://github.com/Pavithra16780/CSA09-192472155-Assignment3.git

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Pair-Work Principle

Both students are responsible for understanding the complete assignment. However, each question contains clearly identified individual responsibilities so that the contribution and understanding of each student can be independently verified.

Student A and Student B must not divide the assignment question-wise. Both students must participate in every question.

The individual responsibilities intentionally alternate across questions so that both students demonstrate evidence of:

Design → Implementation → Analysis → Evaluation

Q1 – Handling Failure in a Bank ATM Simulation **6 Marks**

Bloom's Level: K4 – Analyse

Problem Context – ATM Withdrawal

An ATM simulation reads the account number and withdrawal amount from the user, checks the balance and dispenses cash. Users may type letters instead of numbers, enter

a negative amount, request more than the balance, or the card-reader file may be missing.

1.1 Introduction

An Automated Teller Machine (ATM) is an electronic banking system that allows customers to perform transactions such as withdrawing money, checking account balances, and depositing money. During ATM operations, different errors may occur due to invalid input, insufficient balance, network failure, or hardware problems.

Java exception handling provides a structured way to handle such errors without terminating the entire application unexpectedly. By using try, catch, finally, throw, and throws, an ATM system can identify errors and display suitable messages to users.

1.2 Problem Statement

An ATM system must handle different types of failures during transactions. If a customer enters an invalid amount, requests more money than the available balance, or encounters a network failure, the system should display an appropriate error message.

The objective is to design an exception hierarchy for ATM operations and demonstrate the use of checked and unchecked exceptions.

1.3 Objectives

1. To identify possible failures in ATM transactions.
2. To design suitable custom exception classes.
3. To differentiate checked and unchecked exceptions.
4. To handle exceptions using appropriate catch blocks.
5. To ensure that important cleanup operations are performed using the finally block.

1.4 ATM Exception Hierarchy

The following custom exceptions can be designed:

- ATMException – Base exception for ATM-related failures.
- InvalidAmountException – Raised when the entered amount is zero or negative.

- InsufficientBalanceException – Raised when the withdrawal amount exceeds the available balance.
- NetworkFailureException – Raised when the ATM cannot connect to the bank server.
- CardBlockedException – Raised when the card is blocked.

Exception hierarchy:

Exception

```
 |
 └── ATMException
      |
      ├── InvalidAmountException
      ├── InsufficientBalanceException
      ├── NetworkFailureException
      └── CardBlockedException
```

1.5 Failure and Exception Classification

ATM Failure	**Exception Type**	**Checked/Unchecked**
Invalid withdrawal amount	InvalidAmountException	Checked
Insufficient balance	InsufficientBalanceException	Checked
Network connection failure	NetworkFailureException	Checked
Blocked ATM card	CardBlockedException	Checked

Invalid array or object access	RuntimeException	Unchecked
Invalid numeric input	InputMismatchException	Unchecked

Note: The checked/unchecked classification depends on how the custom exceptions are defined. Custom exceptions extending Exception are checked; those extending RuntimeException are unchecked.

1.6 Catch Block Ordering

Catch blocks must be arranged from the most specific exception to the most general exception.

JAVA:

Example:

```
try {
   // ATM transaction statements
}
catch (InsufficientBalanceException e) {
   System.out.println("Insufficient balance");
}
catch (InvalidAmountException e) {
   System.out.println("Invalid amount");
}
catch (Exception e) {
   System.out.println("General ATM error");
}
```

If a general Exception catch block is placed before a specific exception catch block, the specific catch block becomes unreachable. Java reports a compilation error.

1.7 Role of the Finally Block

The finally block executes after the try and catch blocks in normal exception-handling flow. It is used for cleanup operations such as closing resources and ending a transaction.

JAVA:

```
try {
   System.out.println("ATM transaction started");
}
catch (Exception e) {
   System.out.println("Transaction failed");
}
finally {
   System.out.println("ATM session completed");
}
```

The finally block helps ensure that cleanup code is executed whether an exception occurs or not. It is not an absolute guarantee in situations such as abrupt JVM termination.

1.8 Test Cases

Test Case	**Input/Condition**	**Expected Result**
TC01	Withdraw a valid amount	Transaction successful
TC02	Withdraw more than the balance	Insufficient balance message
TC03	Enter zero amount	Invalid amount message

TC04	Enter a negative amount	Invalid amount message
TC05	Network connection unavailable	Network failure message
TC06	Card is blocked	Card blocked message

1.9 Is a Single catch(Exception e) Acceptable?

Using only one catch(Exception e) block can handle many exceptions, but it does not identify the exact reason for the failure.

For example, an insufficient balance error and a network error may both display the same general message. This makes debugging difficult.

Therefore, specific catch blocks are preferred because they provide meaningful error messages and allow different recovery actions.

1.10 Alternative Input Validation Using Scanner

Instead of allowing invalid input to cause an exception, the program can validate input using Scanner.hasNextInt().

JAVA:

```
import java.util.Scanner;


public class ATMInput {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);


        System.out.print("Enter withdrawal amount: ");


        if (sc.hasNextInt()) {
```

```
            int amount = sc.nextInt();


            if (amount > 0) {
                System.out.println("Valid amount: " + amount);
            } else {
                System.out.println("Amount must be positive");
            }
        } else {
            System.out.println("Please enter a valid number");
        }

        sc.close();
    }
}
```

1.11 Conclusion

Exception handling is important in ATM applications because it prevents unexpected failures from terminating the application. Custom exceptions help identify different transaction problems, while specific catch blocks provide suitable error messages. The finally block supports cleanup operations and improves the reliability of the ATM system.

Q2 – User-Defined Exceptions and Propagation in an Online Examination Portal **7 Marks**

Bloom's Level: K4 – Analyse

Problem Context – Online Examination Portal

An online examination portal must reject a submission if the exam window has closed, if the student is not registered for the course, or if the answer file exceeds the size limit. These rules are checked in different layers: the controller, the service and the storage layer.

2.1 Introduction

An online examination portal allows students to attend examinations, submit answers, and store their responses electronically. The system contains different layers, such as the controller, service, and storage layers.

Errors may occur while validating student details, processing answers, or saving submitted files. Java custom exceptions help identify these errors and communicate them between different layers of the application.

2.2 Problem Statement

The online examination portal must handle different errors during examination submission. For example, a student may submit an invalid answer, the examination session may expire, or the system may fail to store the submitted file.

The system should use custom exceptions to handle these failures and provide meaningful error messages.

2.3 Objectives

1. To create custom exceptions for an online examination portal.
2. To understand checked and unchecked exceptions.
3. To implement exception propagation between application layers.
4. To demonstrate exception chaining using the cause of an exception.
5. To handle storage failures without losing the original error information.

2.4 Application Layers

The online examination portal can be divided into three main layers.

1. Controller Layer

- Receives student requests.
- Accepts examination submission details.
- Sends requests to the service layer.
- Displays success or failure messages.

2. Service Layer

- Validates examination information.
- Checks submission rules.
- Processes student answers.
- Sends the submission to the storage layer.

3. Storage Layer

- Saves answer files.
- Reads stored examination data.
- Handles file-related errors.
- Reports storage failures to the service layer.

2.5 Custom Exception Classes

The following exceptions can be created:

Exception	**Purpose**	**Suggested Type**
InvalidSubmissionException	Invalid examination submission	Checked
ExamSessionExpiredException	Examination time has expired	Checked

SubmissionStorageException	Failure while storing answers	Checked
StudentNotFoundException	Student record is unavailable	Checked
InvalidAnswerException	Invalid answer information	Unchecked or checked, depending on design

2.6 Checked and Unchecked Exceptions

Checked exceptions are checked by the Java compiler. The programmer must handle them using try-catch or declare them using throws.

Examples:

- IOException
- FileNotFoundException
- Custom exceptions extending Exception

Unchecked exceptions occur at runtime and do not require mandatory handling during compilation.

Examples:

- NullPointerException
- ArithmeticException
- IllegalArgumentException
- Custom exceptions extending RuntimeException

Checked exceptions are suitable for expected recoverable failures such as file-storage problems. Unchecked exceptions can be used for programming errors or invalid method arguments.

2.7 Difference Between throw and throws

throw	**throws**

Used to explicitly throw an exception.	Used in a method declaration.
Followed by an exception object.	Followed by exception class names.
Used inside a method or block.	Used in the method signature.
Throws one exception object at a time.	Can declare multiple exception types.

Example:

```
throw new IllegalArgumentException("Invalid answer");
```

```
public void saveSubmission() throws IOException {
    // File storage operation
}
```

2.8 Exception Chaining

Exception chaining is a technique in which one exception is wrapped inside another exception while preserving the original cause.

For example, when a storage operation fails, the storage layer can throw an IOException. The service layer can wrap it inside a custom SubmissionStorageException.

This helps developers understand both the application-level error and the original technical error.

TODO Program – StorageDemo

The assignment requires catching an IOException and throwing a custom exception with the original exception as its cause.

```
import java.io.IOException;


class SubmissionStorageException extends Exception {
    public SubmissionStorageException(String message, Throwable cause) {
        super(message, cause);
```

```
    }
}

class StorageDemo {

    public static void saveSubmission(String fileName)
            throws SubmissionStorageException {

        try {
            if (fileName.equals("answer02.pdf")) {
                throw new IOException("Disk quota exceeded");
            }

            System.out.println("Saved " + fileName);

        } catch (IOException e) {
            throw new SubmissionStorageException(
                "Could not store " + fileName, e
            );
        }
    }

    public static void main(String[] args) {
        try {
            saveSubmission("answer01.pdf");
```

```
            saveSubmission("answer02.pdf");


        } catch (SubmissionStorageException e) {
            System.out.println("Error: " + e.getMessage());
            System.out.println("Cause: " + e.getCause().getMessage());
        }
    }
}
```

Expected Output

Saved answer01.pdf

Error: Could not store answer02.pdf

Cause: Disk quota exceeded

2.9 Exception Propagation

Exception propagation means passing an exception from the method where it occurred to the method that called it.

Example flow:

Controller Layer

    |

    v

Service Layer

    |

    v

Storage Layer

    |

    v

IOException occurs

|

v

SubmissionStorageException

|

v

Service Layer

|

v

Controller handles the error

2.10 Importance of getCause()

The getCause() method returns the original exception that caused the current exception.

If the original cause is not preserved, developers may only see a general message such as "Could not store submission". They may not know whether the problem occurred due to insufficient storage, permission failure, or another file-related error.

2.11 Unchecked Exception Alternative

The custom exception could extend RuntimeException instead of Exception.

This would make it unchecked. However, storage failures are important application errors that usually require explicit handling. Therefore, using a checked exception is suitable when the calling layer must respond to the failure.

2.12 Conclusion

Custom exceptions improve error handling in an online examination portal. Exception propagation allows errors to move between application layers, while exception chaining preserves the original technical cause. This makes the system easier to debug and maintain.

Q3 – Resource Safety, Debugging and Robust Code **7 Marks**

Bloom's Level: K4 – Analyse

Here the responsibilities are deliberately reversed so that the same student does not always perform the design component.

Problem Context – Daily Sales Report Generator

A retail chain generates a daily sales report by reading several store files and writing a summary file. Occasionally the program stops with "Too many open files", and the report is sometimes incomplete.

3.1 Introduction

A daily sales report system reads transaction files, calculates sales information, and generates a report file. During this process, the application uses resources such as file readers, file writers, and log files.

These resources must be closed after use. If they are not closed properly, resource leaks may occur. Java provides manual resource closing and try-with-resources to manage resources safely.

3.2 Problem Statement

A sales report application must read sales data from files and generate a daily report. If an exception occurs during report generation, the program must still release the resources that it has opened.

The objective is to compare manual resource closing with try-with-resources and identify which method provides better resource management.

3.3 Objectives

1. To understand resource management in Java.
2. To identify resource leaks.
3. To compare manual closing and try-with-resources.
4. To understand the importance of closing resources.
5. To handle exceptions during file operations.

3.4 Manual Resource Closing

In manual resource management, the programmer must explicitly close each resource.

Example:

```
import java.io.*;


public class ManualCloseDemo {
    public static void main(String[] args) {
        FileReader reader = null;


        try {
            reader = new FileReader("sales.txt");


            int data;
            while ((data = reader.read()) != -1) {
                System.out.print((char) data);
            }


        } catch (IOException e) {
            System.out.println("File error: " + e.getMessage());


        } finally {
            try {
                if (reader != null) {
                    reader.close();
                    System.out.println("\nFile closed");
```

```
            }
        } catch (IOException e) {
            System.out.println("Error while closing file");
        }
    }
  }
}
```

Explanation:

- The reader is initially assigned null.
- The file is opened inside the try block.
- The file contents are read.
- Any file-related error is handled by the catch block.
- The finally block attempts to close the reader.

Manual closing requires additional code. If several resources are used, each resource must be closed carefully.

3.5 Try-With-Resources

Try-with-resources was introduced in Java 7. It automatically closes resources that implement the AutoCloseable interface.

Example:

```
import java.io.*;


public class TryWithResourcesDemo {
   public static void main(String[] args) {


      try (FileReader reader = new FileReader("sales.txt")) {
```

```
            int data;
            while ((data = reader.read()) != -1) {
                System.out.print((char) data);
            }

        } catch (IOException e) {
            System.out.println("File error: " + e.getMessage());
        }

        System.out.println("\nProgram completed");
    }
}
```

Explanation:

- The file reader is declared inside the try statement.
- Java automatically closes the resource after the try block.
- The programmer does not need to write a separate close statement.
- It reduces the possibility of resource leaks.

3.6 Comparison

Manual Closing	**Try-With-Resources**
Programmer closes resources manually.	Java closes resources automatically.
Requires a finally block for safe closing.	Does not require a separate closing block.
More code is required.	Code is shorter and cleaner.

Closing errors need additional handling.	Closing exceptions are handled by the language mechanism.
More chances of forgetting to close resources.	Reduces the risk of resource leaks.

3.7 Resource Leak

A resource leak occurs when an application opens a resource but fails to release it after use.

Examples include:

- File readers not closed.
- File writers not closed.
- Database connections not released.
- Network sockets left open.

Resource leaks can increase memory and operating-system resource usage. In long-running applications, they may eventually prevent new files or connections from being opened.

3.8 Failure During Report Generation

Suppose a sales report program reads a transaction file and writes the report. If an exception occurs halfway through the process, the output file may be incomplete.

Using try-with-resources ensures that the opened resources are closed even when an exception occurs during processing.

For example:

```
import java.io.*;


public class SalesReport {

   public static void main(String[] args) {


      try (
```

```
        BufferedReader br =
            new BufferedReader(new FileReader("sales.txt"));

        BufferedWriter bw =
            new BufferedWriter(new FileWriter("report.txt"))
    ) {

        String line;

        while ((line = br.readLine()) != null) {
            bw.write(line);
            bw.newLine();
        }

        System.out.println("Report generated successfully");

    } catch (IOException e) {
        System.out.println("Report generation failed");
        System.out.println(e.getMessage());
    }
  }
}
```

3.9 Resource Closing Order

When multiple resources are declared in a try-with-resources statement, Java closes them in the reverse order of their declaration.

For example:

```
try (
    FileReader reader = new FileReader("sales.txt");
    BufferedReader br = new BufferedReader(reader)
) {
    // Processing
}
```

The resources are closed in reverse order: first br, then reader.

3.10 Suppressed Exceptions

If an exception occurs inside the try block and another exception occurs while closing a resource, the closing exception may be recorded as a suppressed exception.

The getSuppressed() method can be used to retrieve suppressed exceptions.

This feature helps preserve the original exception instead of losing important error information.

3.11 Conclusion

Resource management is important in file-processing applications. Manual closing requires careful programming, whereas try-with-resources automatically closes resources. Therefore, try-with-resources is generally preferred for the daily sales report application because it improves reliability, reduces code, and prevents resource leaks.

Practical measurement to include: Record the number of resources opened and closed during your tests. Also record what happens when a file is missing or an exception is deliberately generated. Do not invent these measurements; use your actual program results.

Q4 – Thread Creation and the Thread Life Cycle **7 Marks**

Bloom's Level: K4 – Analyse

Problem Context – Parallel Answer-Script Scanning

An examination cell scans answer scripts from four scanner stations at the same time. Each station is simulated as a thread that processes a batch of scripts and reports the number scanned.

4.1 Introduction

Multithreading is a Java feature that allows multiple threads to execute within a single program. A thread is a lightweight unit of execution.

In a cinema or service-processing system, multiple tasks may need to be processed at the same time. For example, four scanner stations may process four different scripts or jobs simultaneously.

Java supports multithreading through the Thread class and the Runnable interface.

4.2 Problem Statement

A system contains four scanner stations. Each station must process its assigned task. The application should execute the tasks concurrently and demonstrate thread creation, execution, waiting, and termination.

The program must compare the Thread subclass approach and the Runnable interface approach.

4.3 Objectives

1. To understand the concept of multithreading.
2. To create threads using the Thread class.
3. To create threads using the Runnable interface.
4. To understand thread life-cycle methods.
5. To observe concurrent execution and thread scheduling.

4.4 Creating a Thread by Extending the Thread Class

A class can extend the Thread class and override its run() method.

```
class ScannerStation extends Thread {


    ScannerStation(String name) {
        super(name);
    }


    public void run() {
        System.out.println(
            getName() + " started processing"
        );


        for (int i = 1; i <= 3; i++) {
            System.out.println(
                getName() + " processing task " + i
            );
        }


        System.out.println(
            getName() + " completed"
        );
    }
}
```

```
public class ThreadDemo {
    public static void main(String[] args) {

        ScannerStation s1 = new ScannerStation("Station 1");
        ScannerStation s2 = new ScannerStation("Station 2");
        ScannerStation s3 = new ScannerStation("Station 3");
        ScannerStation s4 = new ScannerStation("Station 4");

        s1.start();
        s2.start();
        s3.start();
        s4.start();
    }
}
```

4.5 Creating a Thread Using Runnable

The Runnable interface provides another way to create a thread. The task is written inside the run() method and passed to a Thread object.

```
class ScannerTask implements Runnable {

    String station;

    ScannerTask(String station) {
        this.station = station;
    }
```

```
    public void run() {
        System.out.println(station + " started");

        for (int i = 1; i <= 3; i++) {
            System.out.println(
                station + " processing task " + i
            );
        }

        System.out.println(station + " completed");
    }
}

public class RunnableDemo {
    public static void main(String[] args) {

        Thread t1 = new Thread(new ScannerTask("Station 1"));
        Thread t2 = new Thread(new ScannerTask("Station 2"));
        Thread t3 = new Thread(new ScannerTask("Station 3"));
        Thread t4 = new Thread(new ScannerTask("Station 4"));

        t1.start();
        t2.start();
        t3.start();
        t4.start();
```

```
    }
}
```

4.6 Difference Between Thread and Runnable

Thread Class	Runnable Interface
The class extends Thread.	The class implements Runnable.
The task and thread are combined.	The task is separated from the Thread object.
Java does not support multiple class inheritance.	The class can extend another class and implement Runnable.
Suitable for simple thread examples.	More flexible for reusable tasks.

4.7 Important Thread Methods

Method	Purpose
start()	Starts a new thread and causes its run method to execute.
run()	Contains the task performed by the thread.
sleep()	Pauses the current thread for a specified time.
join()	Makes the calling thread wait for another thread to finish.
isAlive()	Checks whether a thread is still running.
setPriority()	Sets the requested priority of a thread.
getPriority()	Returns the thread priority.

4.8 Difference Between run() and start()

Calling start() creates a new thread of execution. The JVM then invokes the thread's run() method.

Calling run() directly does not create a new thread. It behaves like a normal method call and executes in the current thread.

Therefore, start() should be used when actual multithreading is required.

4.9 Use of Join

The join() method is used when one thread must wait for another thread to complete.

Example:

```
public class JoinDemo {
    public static void main(String[] args)
            throws InterruptedException {

        Thread t1 = new Thread(() -> {
            System.out.println("Task started");
            System.out.println("Task completed");
        });

        t1.start();
        t1.join();

        System.out.println("Main thread completed");
    }
}
```

Expected Output

Task started

Task completed

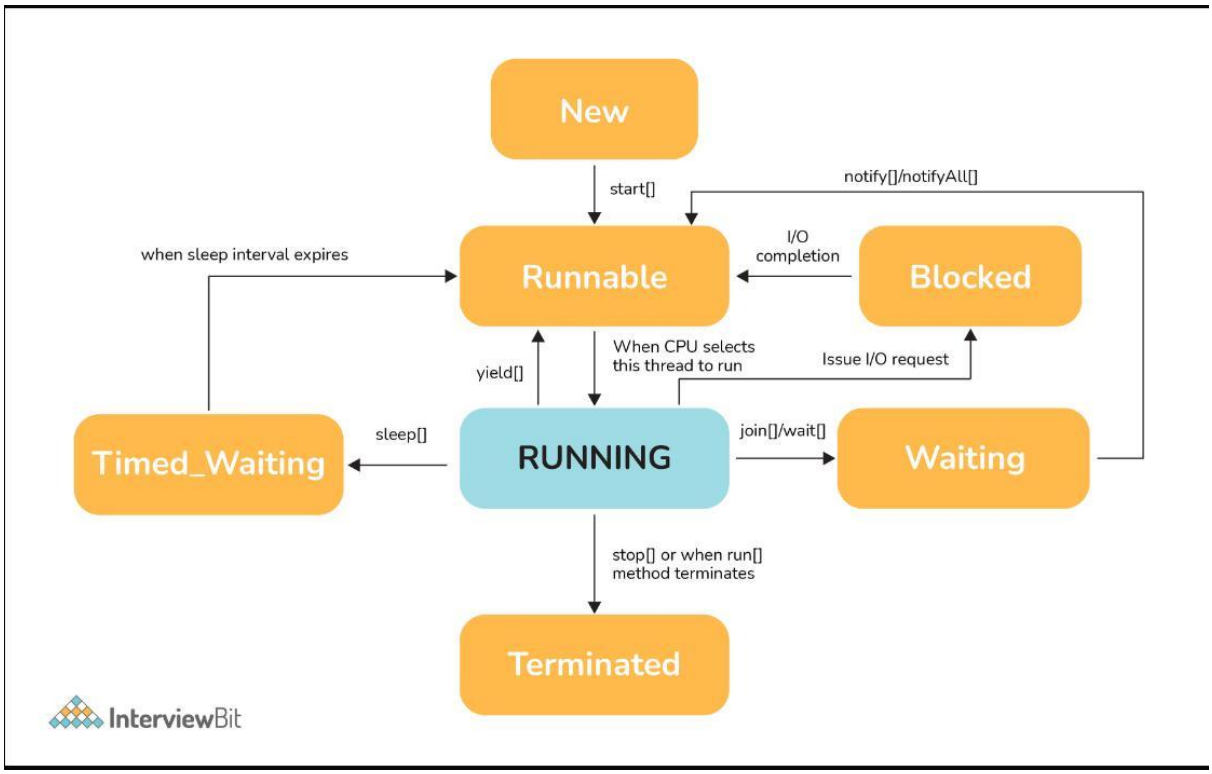
Main thread completed

The main thread prints its completion message after t1 finishes.

4.10 Thread Life Cycle

A Java thread can move through the following states:

1. NEW: The thread object is created but not started.
2. RUNNABLE: The thread is ready to run or running.
3. BLOCKED: The thread is waiting to obtain a monitor lock.
4. WAITING: The thread waits indefinitely for another thread's action.
5. TIMED_WAITING: The thread waits for a specified time.
6. TERMINATED: The thread has completed execution.



4.11 Thread Scheduling and Priority

Thread scheduling is the process of deciding which thread gets CPU execution time.

Java supports thread priorities from 1 to 10:

- Thread.MIN_PRIORITY = 1
- Thread.NORM_PRIORITY = 5
- Thread.MAX_PRIORITY = 10

Priority is only a scheduling hint. It does not guarantee that a higher-priority thread will always execute first.

4.12 Daemon Thread

A daemon thread is a background thread that supports other threads. The JVM can exit when only daemon threads remain.

Example:

```
public class DaemonDemo {
    public static void main(String[] args) {

        Thread t = new Thread(() -> {
            while (true) {
                System.out.println("Background task running");

                try {
                    Thread.sleep(1000);
                } catch (InterruptedException e) {
                    break;
                }
            }
        });

        t.setDaemon(true);
        t.start();

        System.out.println("Main program completed");
    }
}
```

The daemon setting must be applied before the thread is started.

4.13 ExecutorService Alternative

Instead of creating a new thread for every task, Java provides ExecutorService. A fixed thread pool can reuse a limited number of worker threads.

```
import java.util.concurrent.ExecutorService;

import java.util.concurrent.Executors;


public class ExecutorDemo {

   public static void main(String[] args) {


      ExecutorService pool =

         Executors.newFixedThreadPool(4);


      for (int i = 1; i <= 8; i++) {

         final int task = i;


         pool.submit(() -> {

            System.out.println(

               Thread.currentThread().getName()

               + " processing task " + task

            );

         });

      }


      pool.shutdown();
```

```
    }
}
```

If there are n tasks and p worker threads, the tasks may be processed in approximately n/p batches, depending on task duration and scheduling.

4.14 Experiment and Observation Table

Run the scanner program ten times and record the actual execution order.

Run	**First Thread**	**Second Thread**	**Third Thread**	**Fourth Thread**
1	Record result	Record result	Record result	Record result
2	Record result	Record result	Record result	Record result
3	Record result	Record result	Record result	Record result
4	Record result	Record result	Record result	Record result
5	Record result	Record result	Record result	Record result
6	Record result	Record result	Record result	Record result
7	Record result	Record result	Record result	Record result
8	Record result	Record result	Record result	Record result
9	Record result	Record result	Record result	Record result

10	Record result	Record result	Record result	Record result

The execution order may change between runs because thread scheduling is controlled by the JVM and operating system.

4.15 Conclusion

Multithreading improves application responsiveness by allowing multiple tasks to execute concurrently. Java provides the Thread class and Runnable interface for creating threads. Methods such as start(), sleep(), join(), and isAlive() help control thread execution. ExecutorService provides a convenient way to manage multiple tasks using a thread pool.

Q5 – Race Conditions in a Cinema Seat-Booking System **6 Marks**

Bloom's Level: K4 – Analyse

Problem Context – Cinema Seat Booking

A cinema app allows many users to book seats at the same time. Each booking checks whether a seat is free and then marks it as booked. Customers have complained that the same seat was sold twice.

5.1 Introduction

A cinema ticket-booking system allows multiple customers to book seats simultaneously. If two customers attempt to book the same seat at the same time, the system may accidentally assign the same seat to both customers.

This problem occurs due to a race condition. Java synchronization helps prevent multiple threads from modifying shared data incorrectly.

5.2 Problem Statement

A cinema has a limited number of seats. Multiple customers may attempt to book the same seat concurrently. The system must ensure that a seat is booked only once.

The objective is to demonstrate the problem of unsynchronized access and solve it using synchronized methods and blocks.

5.3 Objectives

1. To understand race conditions.
2. To demonstrate incorrect results caused by concurrent access.
3. To implement synchronization using the synchronized keyword.
4. To identify the critical section of a program.
5. To compare synchronization alternatives.

5.4 Race Condition

A race condition occurs when multiple threads access shared data simultaneously and the final result depends on the timing of their execution.

In a seat-booking system, two threads may both check that a seat is available before either thread updates its status. Both threads may then report that the booking is successful.

5.5 Unsynchronized Seat Booking

```
class Cinema {
   boolean booked = false;

   public void book(int seat) {
      if (!booked) {
         System.out.println(
            Thread.currentThread().getName()
            + " booked seat " + seat
         );

         booked = true;
      } else {
         System.out.println("Seat already booked");
      }
   }
}
```

```
public class UnsynchronizedDemo {
    public static void main(String[] args) {

        Cinema cinema = new Cinema();

        Thread t1 = new Thread(
            () -> cinema.book(1), "Customer 1"
        );

        Thread t2 = new Thread(
            () -> cinema.book(1), "Customer 2"
        );

        t1.start();
        t2.start();
    }
}
```

The above program is not thread-safe. Both threads may check the value of booked before it is updated.

5.6 Synchronized Method

The synchronized keyword allows only one thread at a time to execute the synchronized method on the same object.

TODO Program – SeatBooking

The required check-and-book operation must be made atomic using a synchronized method.

```
class SeatBooking {
    boolean[] seats = new boolean[10];

    public synchronized void book(int seat) {
```

```
        if (seat < 0 || seat >= seats.length) {
            System.out.println("Invalid seat number");
            return;
        }

        if (!seats[seat]) {
            seats[seat] = true;
            System.out.println("Seat " + seat + " booked");
        } else {
            System.out.println("Seat already booked");
        }
    }

    public static void main(String[] args) {

        SeatBooking booking = new SeatBooking();

        Thread t1 = new Thread(() -> booking.book(1));
        Thread t2 = new Thread(() -> booking.book(1));

        t1.start();
        t2.start();
    }
}
```

Expected Result: Only one thread can successfully book the same seat. The other thread receives the message Seat already booked.

The assignment's stated expected result for its ten-seat booking test is:

Seats sold: 10

5.7 Synchronized Block

A synchronized block locks only a selected portion of the code.

Example:

```
class SeatManager {
    boolean booked = false;

    public void bookSeat() {

        synchronized (this) {
            if (!booked) {
                booked = true;
                System.out.println("Booking successful");
            } else {
                System.out.println("Seat already booked");
            }
        }
    }
}
```

A synchronized block can reduce the amount of code protected by a lock. It is useful when only a small part of a method needs synchronized access.

5.8 Synchronized Method vs Synchronized Block

Synchronized Method	**Synchronized Block**
Locks the complete synchronized method.	Locks only the selected code block.
Simple to implement.	Provides more control over locking.
May hold the lock for longer.	Can reduce lock duration.
Uses the object's monitor for an instance method.	Can specify the object used for locking.

5.9 Static Synchronization

A static synchronized method locks the class-level monitor rather than an individual object monitor.

It is useful when multiple objects of the same class must coordinate access to shared static data.

Example:

```
class CinemaCounter {
   static int totalSeats = 10;

   public static synchronized void sellSeat() {
      if (totalSeats > 0) {
         totalSeats--;
         System.out.println("Seat sold");
      }
   }
}
```

5.10 Synchronization Alternatives

1. AtomicBoolean: Can be used for atomic seat availability updates.
2. ReentrantLock: Provides explicit lock and unlock operations.
3. ConcurrentHashMap: Can be used for concurrent key-value operations. An atomic operation such as putIfAbsent() can help prevent duplicate seat assignments when the seat identifier is the key.

5.11 Experiment and Observation

Run the booking program multiple times and compare the unsynchronized and synchronized versions.

Measurement	**Unsynchronized**	**Synchronized**
Same seat booked by multiple threads	May occur	Prevented when all access uses the same lock
Data consistency	Not guaranteed	Protected
Lock overhead	No explicit synchronized lock	Locking overhead exists

Correct booking result	Not guaranteed	Guaranteed for the protected operation

For a timing comparison, execute the same number of booking operations in both versions and record the actual time measured on your computer.

5.12 Application with Multiple Screens

For a cinema containing 20 screens, each screen can have its own seat-booking object and lock. This allows customers to book seats on different screens without unnecessarily blocking each other.

5.13 Conclusion

Synchronization is necessary when multiple threads access shared data. In a cinema booking system, synchronized methods and blocks prevent race conditions and duplicate bookings. The correct synchronization method helps maintain data consistency and reliable seat allocation.

Q6 – Deadlock and Inter-Thread Communication **7 Marks**

Bloom's Level: K4 – Analyse

Problem Context – Hostel Wallet Transfers and Mess Token Counter

Part A: Students transfer money between hostel wallet accounts; each transfer locks both accounts. Part B: A mess counter prepares meal tokens (producer) and students collect them (consumers) from a counter that can hold at most 5 tokens.

6.1 Introduction

Multithreaded applications often use shared resources. When multiple threads attempt to access the same resources, problems such as deadlock and race conditions may occur.

Java provides synchronization, wait(), notify(), and notifyAll() to coordinate threads. The producer-consumer problem is a common example of thread communication using a shared buffer.

6.2 Problem Statement

A wallet application allows users to transfer money between accounts. If two threads transfer money in opposite directions and lock accounts in different orders, deadlock may occur.

Another application involves a producer generating tokens and a consumer collecting them using a bounded buffer with a maximum capacity of five.

The objective is to demonstrate deadlock, prevent it using a consistent lock order, and implement producer-consumer communication.

6.3 Objectives

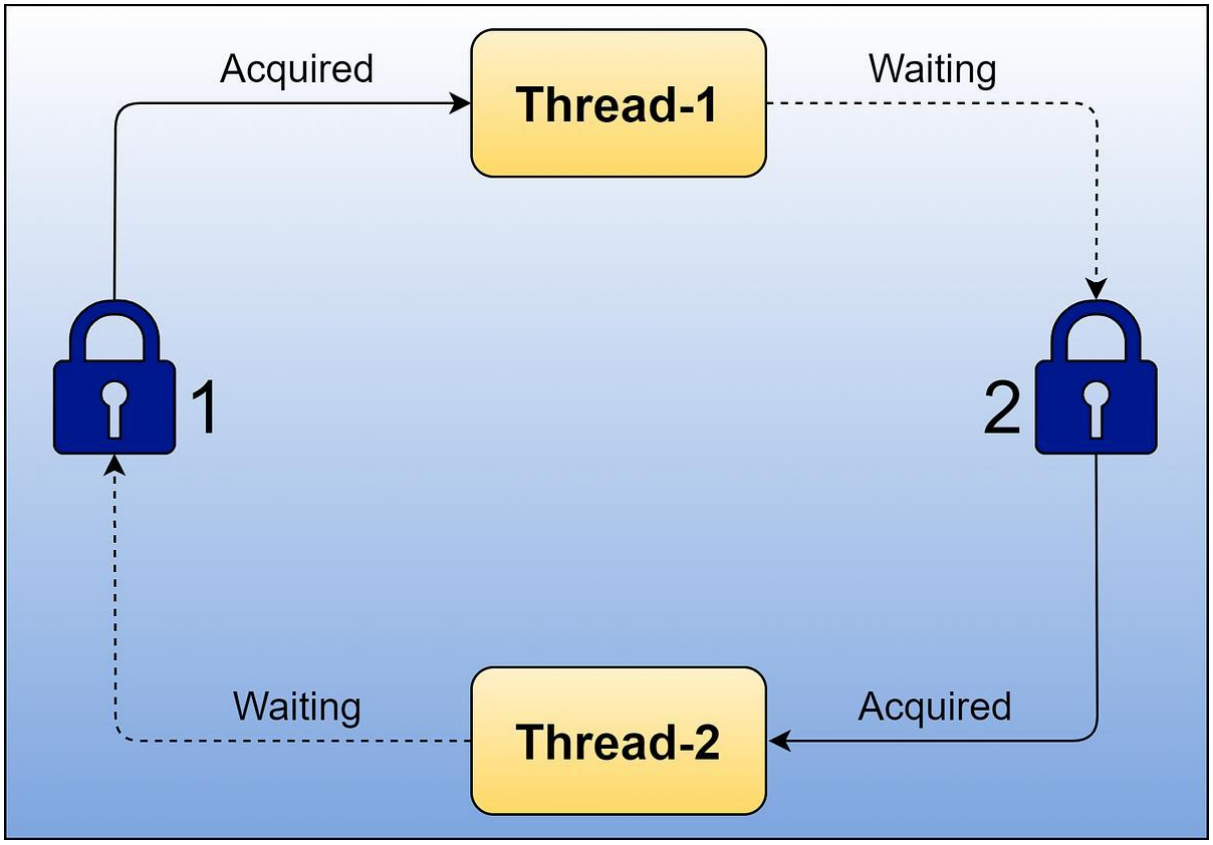
1. To understand deadlock in multithreaded programs.
2. To identify the four necessary conditions for deadlock.
3. To prevent deadlock using consistent lock ordering.
4. To understand wait(), notify(), and notifyAll().
5. To implement a bounded buffer using a circular array.

6.4 Wallet Transfer Deadlock

Suppose two accounts, A and B, are shared by two threads.

- Thread 1 locks account A and then waits for account B.
- Thread 2 locks account B and then waits for account A.

Both threads wait indefinitely. This situation is called deadlock.



6.5 Four Conditions for Deadlock

Deadlock can occur when all four of the following conditions exist:

1. Mutual Exclusion: A resource can be held by only one thread at a time.
2. Hold and Wait: A thread holds one resource while waiting for another.
3. No Preemption: A resource cannot be forcibly taken away from a thread.
4. Circular Wait: A group of threads wait for resources held by one another in a circular order.

6.6 Deadlock Prevention Using Consistent Lock Ordering

A common solution is to acquire locks in a consistent order.

For example, every wallet transfer should lock the account with the smaller account ID first and then lock the account with the larger account ID.

This prevents two threads from acquiring the same pair of locks in opposite orders.

Example:

class Wallet {

```
    int balance;
    int id;

    Wallet(int id, int balance) {
        this.id = id;
        this.balance = balance;
    }
}

class Transfer {

    public static void transfer(
            Wallet a, Wallet b, int amount) {

        Wallet first = a.id < b.id ? a : b;
        Wallet second = a.id < b.id ? b : a;

        synchronized (first) {
            synchronized (second) {

                if (a.balance >= amount) {
                    a.balance -= amount;
                    b.balance += amount;

                    System.out.println("Transfer successful");
```

```
            } else {
                System.out.println("Insufficient balance");
            }
        }
    }
}
```

This example demonstrates lock ordering. A complete application should also validate the amount and account information.

6.7 Producer-Consumer Problem

The producer-consumer problem contains two types of threads:

Producer:

- Generates data.
- Places data into a shared buffer.
- Waits when the buffer is full.

Consumer:

- Removes data from the buffer.
- Processes the data.
- Waits when the buffer is empty.

The buffer has a fixed capacity of five.

6.8 Use of wait(), notify(), and notifyAll()

Method	**Purpose**
wait()	Makes the current thread wait and releases the object's monitor.
notify()	Wakes one thread waiting on the same object's monitor.
notifyAll()	Wakes all threads waiting on the same object's monitor.

These methods must be called while the thread owns the object's monitor, normally inside a synchronized method or block.

6.9 TODO Program – TokenCounter

The assignment requires a bounded buffer of size five. The following implementation demonstrates the required circular-buffer operations.

```
class TokenCounter {

   private int[] buffer = new int[5];

   private int in = 0;
   private int out = 0;
   private int count = 0;

   public synchronized void put(int value)
         throws InterruptedException {

      while (count == 5) {
         wait();
      }

      buffer[in] = value;
      in = (in + 1) % 5;
      count++;

      notifyAll();
```

```
    }

    public synchronized int take()
            throws InterruptedException {

        while (count == 0) {
            wait();
        }

        int value = buffer[out];
        out = (out + 1) % 5;
        count--;

        notifyAll();

        return value;
    }
}
```

Explanation of the put operation:

1. The producer checks whether the buffer is full.
2. If the buffer is full, it waits.
3. The producer stores the value at the in position.
4. The in position is updated using circular indexing.
5. The count is increased.
6. notifyAll() wakes waiting threads.

Explanation of the take operation:

1. The consumer checks whether the buffer is empty.
2. If the buffer is empty, it waits.
3. The consumer reads the value at the out position.
4. The out position is updated.
5. The count is decreased.
6. notifyAll() wakes waiting threads.

6.10 Why Use While Instead of If?

A while loop is preferred because a waiting thread may wake up even when the required condition is not yet satisfied. This is called a spurious wakeup. Another thread may also change the buffer before the awakened thread reacquires the lock.

Therefore, the condition must be checked again after waking up.

6.11 Difference Between Sleep and Wait

sleep()	**wait()**
Pauses the current thread for a specified time.	Waits for another thread's notification or interruption.
Does not release the monitor lock.	Releases the monitor lock of the object.
Belongs to the Thread class.	Belongs to the Object class.
Can be used without synchronized code.	Must be called while owning the object's monitor.

6.12 ArrayBlockingQueue Alternative

Java provides ArrayBlockingQueue, which is a bounded blocking queue.

It can be used instead of manually implementing the producer-consumer buffer.

```
import java.util.concurrent.ArrayBlockingQueue;

import java.util.concurrent.BlockingQueue;
```

```
public class QueueDemo {
    public static void main(String[] args) {

        BlockingQueue<Integer> queue =
            new ArrayBlockingQueue<>(5);

        try {
            queue.put(10);
            queue.put(20);

            System.out.println(queue.take());
            System.out.println(queue.take());

        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

The put() method waits if the queue is full, and take() waits if the queue is empty.

6.13 Expected Output

The assignment specifies the following expected result for the token collection test:

Sum of tokens collected: 210 (expected 210)

6.14 Conclusion

Deadlock is a serious problem in multithreaded applications because it prevents threads from completing their tasks. Consistent lock ordering helps prevent circular waiting.

The producer-consumer problem demonstrates how wait() and notifyAll() coordinate threads. A bounded buffer and ArrayBlockingQueue provide ways to manage shared data safely.

Q7 – Design a Concurrent, Fault-Tolerant Processor from First Principles 10 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Semester Result Processing

The examination cell must process 10,000 answer-script score files at the end of the semester. Each file must be read, validated and used to compute the student's result. Some files are corrupt or missing. The processing must finish quickly, a single bad file must not stop the whole run, and a summary of successes and failures must be produced at the end.

You are not given the design. You must develop it from the requirements.

7.1 Introduction

A concurrent fault-tolerant processor is a system that processes multiple files simultaneously while handling errors without stopping the entire application.

In an educational institution, examination results may be stored in thousands of score files. Processing each file one by one can take more time. Java provides multithreading and ExecutorService to process multiple files concurrently.

The system should continue processing valid files even when some files are missing or corrupted.

7.2 Problem Statement

An institution has 10,000 student score files that must be processed. Each file contains student marks and related information.

The system must read the files, calculate the required statistics, and generate a summary. Some files may be missing or corrupted. Therefore, the processor must handle file-related exceptions and continue processing the remaining valid files.

7.3 Objectives

1. To develop a concurrent file-processing application.
2. To use ExecutorService for managing multiple threads.
3. To implement Callable and Future.
4. To handle missing and corrupted files.
5. To compare execution time using different thread-pool sizes.
6. To generate a final summary after processing all files.

7.4 System Requirements

Requirement	Description
Programming Language	Java
Processing Framework	ExecutorService
Task Interface	Callable
Result Handling	Future
Data Storage	Text or CSV files
Input Files	10,000 score files
Thread Pool Sizes	1, 2, 4 and 8
Error Handling	Exception handling
Output	Processed records and summary

7.5 System Architecture

The system contains the following components:

1. File Input Module

- Reads the score files.
- Identifies available files.
- Passes file names to the processing module.

2. Concurrent Processing Module

- Creates worker threads.

- Assigns files to worker threads.
- Processes multiple files simultaneously.

3. Exception Handling Module

- Detects missing files.
- Handles corrupted files.
- Records processing errors.
- Allows other valid tasks to continue.

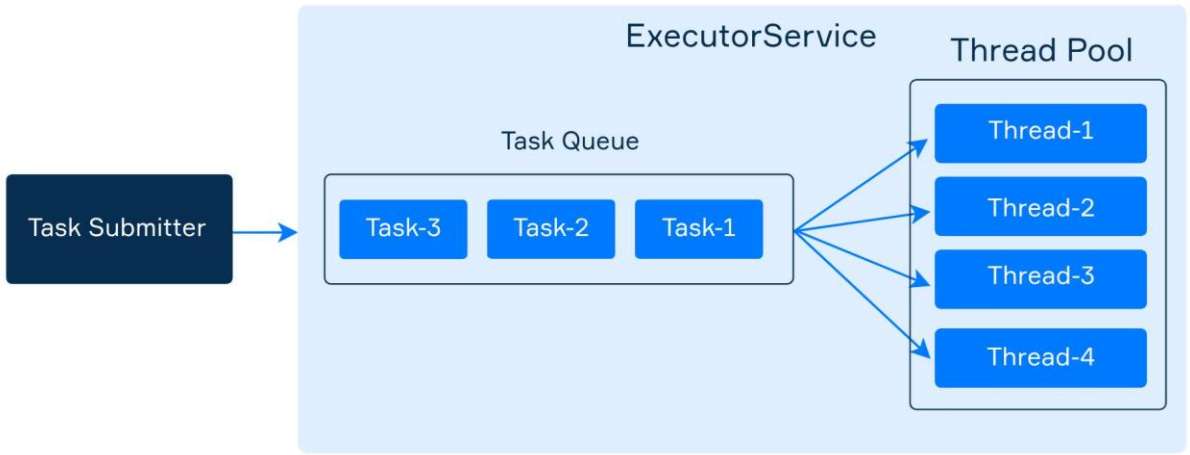
4. Result Collection Module

- Collects results using Future.
- Combines valid processing results.
- Generates the final summary.

5. Performance Measurement Module

- Measures total execution time.
- Compares different thread-pool sizes.
- Identifies a suitable configuration.

7.6 System Architecture Diagram



7.7 Stage 1 – Joint Requirements

Both students should discuss and finalize the requirements before implementation.

The system must:

1. Process 10,000 score files.
2. Use ExecutorService to manage worker threads.
3. Use Callable to perform file-processing tasks.
4. Use Future to retrieve processing results.
5. Handle missing and corrupted files.
6. Collect valid results.
7. Measure execution time for different thread-pool sizes.
8. Generate a final summary.

7.8 Stage 2 – Student A: ExecutorService Implementation

Student A is responsible for implementing the concurrent processing structure.

Responsibilities:

- Create an ExecutorService.
- Select a fixed thread pool.
- Create Callable tasks.
- Submit tasks to the executor.
- Collect results using Future.
- Shut down the executor after processing.

Sample Java Program

```
import java.util.*;

import java.util.concurrent.*;

import java.io.*;
```

```
class ScoreProcessor implements Callable<Integer> {

    private String fileName;

    ScoreProcessor(String fileName) {
        this.fileName = fileName;
    }

    public Integer call() throws Exception {

        int total = 0;

        try (Scanner sc = new Scanner(new File(fileName))) {

            while (sc.hasNextInt()) {
                total += sc.nextInt();
            }

            return total;
        }
    }
}

public class ConcurrentProcessor {
```

```
public static void main(String[] args) {

    ExecutorService pool =
        Executors.newFixedThreadPool(4);

    List<Future<Integer>> results = new ArrayList<>();

    for (int i = 1; i <= 10; i++) {

        String fileName = "score" + i + ".txt";

        results.add(
            pool.submit(new ScoreProcessor(fileName))
        );
    }

    int grandTotal = 0;
    int successfulFiles = 0;
    int failedFiles = 0;

    for (Future<Integer> result : results) {

        try {
            int value = result.get();
```

```
                grandTotal += value;
                successfulFiles++;

            } catch (Exception e) {
                failedFiles++;
                System.out.println(
                    "File processing failed: " + e.getMessage()
                );
            }
        }

        pool.shutdown();

        System.out.println("Successful files: " + successfulFiles);
        System.out.println("Failed files: " + failedFiles);
        System.out.println("Grand total: " + grandTotal);
    }
}
```

Note: This is a sample implementation using ten file names to demonstrate the design. For the assignment's full experiment, change the file-generation and processing setup to cover all 10,000 files.

7.9 Stage 3 – Student B: Testing and Fault Handling

Student B is responsible for testing the application and checking its fault-handling capability.

The following conditions must be tested:

1. A valid score file.

2. A missing score file.
3. An empty score file.
4. A corrupted score file.
5. A file containing invalid data.
6. Multiple files processed simultaneously.

Testing Table

Test Case	**Input Condition**	**Expected Behaviour**
TC01	Valid score file	File processed successfully
TC02	Missing file	Error recorded; other files continue
TC03	Empty file	Empty-file result handled
TC04	Corrupted file	Processing error recorded
TC05	Invalid score data	Invalid data handled
TC06	Multiple valid files	Files processed concurrently

7.10 Stage 4 – Performance Comparison

The assignment requires measuring execution time with different thread-pool sizes.

Use the same input files and the same processing operation for every test.

Thread Pool Size	**Number of Files**	**Actual Execution Time**
1	10,000	Record measured time
2	10,000	Record measured time
4	10,000	Record measured time
8	10,000	Record measured time

Performance Analysis

- A pool size of 1 processes tasks using one worker thread.
- A pool size of 2 allows two tasks to execute concurrently.
- A pool size of 4 allows up to four worker threads to execute tasks concurrently.

- A pool size of 8 allows up to eight worker threads to execute tasks concurrently.
- Increasing the number of threads may improve performance, but it does not guarantee faster execution.

The actual result depends on the computer's CPU, available memory, storage speed, file sizes, and workload.

7.11 Why More Threads Do Not Always Improve Performance

Increasing the thread count does not always reduce execution time because:

1. Threads require memory for their execution stacks.
2. Too many threads increase context switching.
3. Multiple threads may compete for CPU resources.
4. File operations may be limited by storage speed.
5. Thread synchronization can create additional overhead.
6. The available number of CPU cores limits parallel processing benefits for CPU-intensive tasks.

Therefore, the thread-pool size should be selected based on experimental results.

7.12 Importance of Future

The Future interface represents the result of an asynchronous task.

Important methods include:

Method	Purpose
get()	Retrieves the result and waits if necessary.
isDone()	Checks whether the task is completed.
isCancelled()	Checks whether the task was cancelled.
cancel()	Attempts to cancel the task.

7.13 Fault Tolerance

Fault tolerance means that the application can continue operating even when some operations fail.

In this project:

- Missing files are recorded as failures.
- Corrupted files are handled using exception handling.
- Valid files continue to be processed.
- The application collects successful results.
- The final summary reports both successful and failed processing operations.

A fault-tolerant application should not silently ignore failures. It should record the file name and error details so that the problem can be investigated.

7.14 Alternative Approaches

Parallel Streams

Parallel streams can process collections using multiple threads. They are useful for simpler data-processing operations.

One Thread Per File

A separate thread can be created for each file. However, creating 10,000 individual threads can consume significant resources.

ExecutorService

ExecutorService is generally more suitable because it reuses a limited number of worker threads.

7.15 Complexity Analysis

Let:

- n = number of files.
- p = number of worker threads.

The application must process all n files. The total processing work depends on the number of records and the size of each file.

The executor limits the number of simultaneously active worker threads to approximately p. The time taken depends on the processing workload, file size, CPU resources, and storage performance.

Memory usage includes the executor, task objects, result objects, and any data stored during processing. Storing all results in memory may require additional memory for large workloads.

7.16 Conclusion

A concurrent fault-tolerant processor improves the efficiency of large-scale file processing. ExecutorService manages worker threads, Callable performs individual tasks, and Future retrieves their results. Exception handling allows the application to continue processing valid files when some files are missing or corrupted. Performance measurements help select a suitable thread-pool size.

OVERALL CONCLUSION

This assignment demonstrates important Java programming concepts related to exception handling, resource management, multithreading, synchronization, deadlock prevention, and concurrent processing.

The ATM application demonstrates custom exception handling and the use of specific catch blocks. The online examination portal demonstrates exception propagation and exception chaining. The daily sales report application compares manual resource closing with try-with-resources.

The scanner station application explains thread creation and thread life-cycle methods. The cinema booking application demonstrates race conditions and synchronization. The wallet transfer and producer-consumer applications explain deadlock prevention and thread communication.

Finally, the concurrent fault-tolerant processor demonstrates how ExecutorService, Callable, and Future can be used to process multiple files while handling failures.

Through these questions, the assignment develops practical knowledge of Java exception handling and concurrent programming. It also improves debugging, testing, GitHub usage, and teamwork skills.

MARK DISTRIBUTION

Question	Assessed Area	Bloom's Level	Marks
Q1	Exception hierarchy, try, catch and finally	K4 – Analyse	6
Q2	User-defined exceptions, propagation and chaining	K4 – Analyse	7
Q3	try-with-resources and debugging	K4 – Analyse	7
Q4	Thread creation and life cycle	K4 – Analyse	7
Q5	Race conditions and synchronization	K4 – Analyse	6
Q6	Deadlock and inter-thread communication	K4 – Analyse	7
Q7	Concurrent, fault-tolerant design from first principles	K5 – Evaluate	10
	TOTAL		**50**